

RecruitFlow

Job Recruitment & Interview Management Platform

Field	Details
Document	Product Requirements Document (PRD) & User Stories
Project	RecruitFlow: Job Recruitment & Interview Management Platform

1. Introduction & Purpose

RecruitFlow is a microservices-based application that lets a recruiting team manage the full hiring lifecycle: posting jobs, receiving and reviewing applications, scheduling interviews, and recording hiring decisions. It replaces the spreadsheets and email threads many small teams rely on with a single system of record for recruiters, candidates, and interviewers.

2. Project Overview

RecruitFlow answers a small set of operational questions that any recruiting team needs answered at all times: which jobs are open, who has applied, where each application sits in the pipeline, and how each candidate performed at interview. The system is built as six Spring Boot microservices behind an API Gateway, with a React frontend serving role-based views for Candidates, Recruiters, and Interviewers.

3. Objectives

- Provide a single system of record for every job posting, application, and interview outcome.
- Let candidates self-serve: search jobs, apply, and track application status without contacting a recruiter directly.
- Give recruiters a complete pipeline view: review applications, shortlist candidates, schedule interviews, and record decisions.
- Give interviewers a simple way to submit structured, comparable feedback on every candidate they see.
- Demonstrate a production-style microservices architecture: API Gateway, service discovery, and clean REST APIs.

4. Scope

- Candidate, Recruiter, and Interviewer roles with role-based access.
- Job posting, search, and lifecycle management (Draft, Open, Closed).
- Application submission, status tracking, and withdrawal (including a WITHDRAWN status).
- Interview scheduling and structured feedback capture.
- Final hiring decisions (selected, rejected, hired).
- In-app notifications for key status changes, addressed to any user role.

Out of scope:

- Video interviews and calendar integrations (Zoom, Google Calendar).
- Real email/SMS delivery — notifications are in-app only.
- AI-based resume matching or parsing.
- OAuth/social login.

- Recruiter/company verification before a job can be published — Admin currently moderates at the user-account level (suspend/reactivate), not at the individual job level.
- A dedicated Interviewer login/role experience — the INTERVIEWER role exists in the data model but there is no separate interviewer-facing flow distinct from the Recruiter's Feedback screen.

5. User Roles

Role	Key Capabilities
Candidate	Browse and search jobs, apply, track application status, view interview schedule and notifications.
Recruiter	Post and manage jobs, review and shortlist applications, schedule interviews, make final hiring decisions, view dashboard.
Admin	Platform-level account moderation: view all users, suspend/reactivate accounts.

6. Services

RecruitFlow is decomposed into six independently deployable Spring Boot microservices, each owning its own database. All are registered with Eureka and are reachable only through the API Gateway.

Service	Responsibility & Owned Data
Auth Service	Registration, login, JWT issuance, and role validation for all users. Owns the Users table.
Job Service	Create, edit, and close job postings; search by skill, location, and experience. Owns the Jobs table.
Profile Service	Candidate and recruiter profile data — resume, skills, department, designation. Owns the Candidates and Recruiters tables.
Application Service	Apply, withdraw, and the full application status workflow (APPLIED through HIRED/WITHDRAWN); recruiter review, shortlist, and reject actions. Owns the Applications table.
Interview Service	Interview scheduling and structured feedback capture (technical, communication, problem-solving scores). Owns the Interviews table.
Notification Service	Generates and stores in-app notifications whenever an application or interview status changes, addressed to the relevant user. Owns the Notifications table.

Two supporting components sit alongside these six but are not domain services: the API Gateway (routing, auth enforcement, rate limiting) and the Eureka Server (service registry and discovery).

7. Functional Requirements

7.1 Authentication & Access Control

ID	Requirement
FR-1	A user shall be able to register as CANDIDATE or RECRUITER with email and password. Passwords are stored as BCrypt hashes, never plaintext.

FR-2	On successful login, the system shall issue a JWT containing the user's id, email, and role, valid for 1 hour.
FR-3	The API Gateway shall reject any request lacking a valid, unexpired JWT, except /auth/register and /auth/login.
FR-4	The API Gateway shall reject any request to a path containing /admin/ unless the JWT's role claim is ADMIN, regardless of which downstream service owns the endpoint.
FR-5	A disabled (suspended) user's login attempt shall be rejected with a clear message, even with correct credentials.
FR-6	Any authenticated user may retrieve another user's basic, non-sensitive info (id, email, role) by id — used to display candidate identity to a reviewing recruiter.

7.2 Admin

ID	Requirement
FR-7	Admin shall be able to list all registered users across the platform.
FR-8	Admin shall be able to suspend a user account, immediately blocking future logins.
FR-9	Admin shall be able to reactivate a previously suspended account.

7.3 Job Management

ID	Requirement
FR-10	A Recruiter shall be able to create a job posting (title, description, skills, location, minimum experience). New postings default to DRAFT status.
FR-11	A Recruiter shall be able to edit a job only while it is in DRAFT status; edits to a published (OPEN) or CLOSED job shall be rejected.
FR-12	A Recruiter shall be able to publish (DRAFT → OPEN) or close (→ CLOSED) any job they own.
FR-13	A Candidate shall be able to search jobs by skill, location, and minimum experience, using combinable, partial-match filters. Only OPEN jobs are returned.
FR-14	A Recruiter shall be able to view all jobs they have posted, regardless of status, for management purposes.

7.4 Profiles

ID	Requirement
FR-15	A Candidate shall be able to create and update a profile: resume URL, skills, contact number, location.
FR-16	A Recruiter shall be able to create and update a profile: company, department, designation.

7.5 Applications & Status Workflow

ID	Requirement
FR-17	A Candidate shall be able to apply to an OPEN job. A second application by the same candidate to the same job shall be rejected as a duplicate.
FR-18	Application status shall follow a guarded state machine: APPLIED → UNDER_REVIEW → SHORTLISTED → INTERVIEW_SCHEDULED → SELECTED or REJECTED → HIRED, with WITHDRAWN reachable only from APPLIED.
FR-19	Any transition attempted from a status that does not permit it shall be rejected with a descriptive error identifying the current status — never silently accepted or ignored.
FR-20	A Candidate shall be able to view all of their own applications and each one's current status.
FR-21	A Recruiter shall be able to view all applications submitted to a specific job they posted.

7.6 Interviews

ID	Requirement
FR-22	A Recruiter shall be able to schedule an interview directly from the application review screen for any SHORTLISTED application, without needing to know or enter an application ID. Scheduling both creates the interview record and advances the application to INTERVIEW_SCHEDULED.
FR-23	Feedback (technical score, communication score, comments) shall be submittable against any INTERVIEW_SCHEDULED application, selected by candidate name rather than a raw identifier.
FR-24	Submitted feedback shall be retrievable per application (GET /interviews/{applicationId}) and displayed to the reviewing recruiter.

7.7 Notifications

ID	Requirement
FR-25	The system shall create a notification for the affected candidate when their application is shortlisted, rejected, or withdrawn.
FR-26	A failure to deliver a notification (e.g. notification-service unavailable) shall be logged and shall not roll back the triggering status change.
FR-27	A user shall be able to view their notifications and mark individual notifications as read.

8. Non-Functional Requirements

ID	Requirement
NFR-1	Every service shall return a consistent JSON error shape (timestamp, status, error, message, path) via a shared exception-handling pattern, rather than framework-default error pages or bare strings.
NFR-2	Each service shall own an independent PostgreSQL schema; no service shall query another service's database directly.
NFR-3	Services shall register with and be discoverable through Eureka rather than hardcoded host/port references between services.
NFR-4	Passwords shall never be stored or logged in plaintext.
NFR-5	The frontend shall not require the user to manually type or know an internal numeric ID (application ID, job ID) to perform any workflow action — all such actions shall be reachable through named, contextual UI (candidate email, job title).
NFR-6	The core status-guard logic (JobApplicationService) shall have automated test coverage for both the success and rejection path of every transition.

9. User Stories & Acceptance Criteria

FIELD	CONTENT
Story ID	US-001
Title	User Registration
As a...	New user
I want to...	register with my email, password, and role
So that...	I can access the platform according to my role
Acceptance Criteria	1. Given valid registration details, when I submit the form, then my account is created successfully. 2. Given an email already exists, when I register, then the system displays a duplicate email error. 3. Given required fields are missing, when I submit the form, then validation errors are displayed.
Priority	High

FIELD	CONTENT
Story ID	US-002
Title	User Login
As a...	Registered user
I want to...	log in using my email and password

FIELD	CONTENT
So that...	I can securely access the platform
Acceptance Criteria	1. Given valid credentials, when I log in, then a JWT token is generated. 2. Given invalid credentials, when I log in, then an error message is displayed. 3. Given a valid token, when I access protected APIs, then access is granted.
Priority	High

FIELD	CONTENT
Story ID	US-003
Title	Create Job Posting
As a...	Recruiter
I want to...	create a new job posting
So that...	candidates can apply for available positions
Acceptance Criteria	1. Given valid job details, when I submit, then the job is created successfully. 2. Given mandatory fields are missing, when I submit, then validation errors are shown. 3. Given a job was just created, when candidates browse listings, then the DRAFT job does not appear in candidate search.
Priority	High

FIELD	CONTENT
Story ID	US-004
Title	Search Jobs
As a...	Candidate
I want to...	search jobs using skills, location, or experience
So that...	I can find suitable opportunities quickly
Acceptance Criteria	1. Given search filters, when I search, then matching jobs are displayed. 2. Given multiple filters, when I search, then results satisfy all filters. 3. Given no matching jobs, when I search, then an empty result message is shown.
Priority	High

FIELD	CONTENT
Story ID	US-005
Title	Apply for Job
As a...	Candidate
I want to...	apply for an available job
So that...	recruiters can review my application
Acceptance Criteria	1. Given an open job, when I apply, then an application is created successfully. 2. Given I already applied, when I apply again, then the system prevents duplicate applications. 3. Given a new application, when it is created, then its status is initially set to APPLIED.
Priority	High

FIELD	CONTENT
Story ID	US-006
Title	View Application Status
As a...	Candidate
I want to...	view the status of my applications
So that...	I can track my recruitment progress
Acceptance Criteria	1. Given I have submitted applications, when I open my applications view, then all of them are displayed with their current status. 2. Given an application is in APPLIED status, when I withdraw it, then its status becomes WITHDRAWN. 3. Given an application is SHORTLISTED or beyond, when I try to withdraw it, then withdrawal is not allowed. 4. Given a status changes, when I refresh the view, then the updated status is reflected immediately.
Priority	High

FIELD	CONTENT
Story ID	US-007
Title	Review Applications
As a...	Recruiter
I want to...	review, shortlist, or reject applications

FIELD	CONTENT
So that...	qualified candidates move to the next stage
Acceptance Criteria	1. Given a job I posted, when I open it, then I can view all applications submitted for it. 2. Given an application under review, when I shortlist it, then its status updates and the candidate receives a notification. 3. Given an application under review, when I reject it, then its status updates and the candidate receives a notification.
Priority	High

FIELD	CONTENT
Story ID	US-008
Title	Schedule Interview
As a...	Recruiter
I want to...	schedule interviews for shortlisted candidates
So that...	the interview process can begin
Acceptance Criteria	1. Given a candidate is not shortlisted, when I try to schedule an interview, then the system blocks it. 2. Given a shortlisted candidate, when I schedule an interview, then the date, time are saved successfully. 3. Given an interview is scheduled, when it is saved, then the candidate's application status changes to INTERVIEW_SCHEDULED.
Priority	High

FIELD	CONTENT
Story ID	US-009
Title	Submit Interview Feedback
As a...	Recruiter
I want to...	submit interview feedback
So that...	recruiters can make informed hiring decisions
Acceptance Criteria	1. Given a completed interview, when I submit feedback, then technical, communication, and problem-solving scores are recorded. 2. Given feedback is submitted, when I save it, then my comments are stored alongside the scores. 3. Given feedback already exists for an interview, when I submit again, then it updates the existing record rather than creating a duplicate.

FIELD	CONTENT
Priority	High

FIELD	CONTENT
Story ID	US-010
Title	Final Hiring Decision
As a...	Recruiter
I want to...	mark candidates as selected or rejected
So that...	the recruitment process reaches completion
Acceptance Criteria	1. Given interview feedback has been reviewed, when I mark a candidate SELECTED , then the application status updates and the candidate receives an offer notification. 2. Given interview feedback has been reviewed, when I mark a candidate REJECTED , then the application status updates and the candidate receives a rejection notification. 3. Given a decision is made, when it is saved, then it is reflected immediately on both the candidate's and recruiter's dashboards.
Priority	High

FIELD	CONTENT
Story ID	US-011
Title	Suspend/Reactivate User
As a...	Admin
I want to...	Suspend a misbehaving account and reactivate it later
So that...	I can moderate platform access
Acceptance Criteria	1. Given a valid user ID, when I suspend the user, then subsequent login attempts are rejected even with correct credentials. 2. Given a suspended account, when I reactivate the user, then the user can log in successfully again. 3. Given a non-admin token, when it is used against an Admin endpoint, then the request is rejected with 403 Forbidden .
Priority	High